03

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

Nest CLI와 프로젝트 구조

제1부 기반: 백엔드와 Node.js 런타임 · 1장 오리엔테이션

nest new가 만든 파일의 역할, 커진 프로젝트를 나누는 법



© 한상훈 NestJS 백엔드 마스터

Express 프로젝트 세 개

프로젝트	라우터 위치	DB 연결 코드
첫 번째	`routes/`	`db.js`
두 번째	`api/`	`app.js` 맨 위
세 번째	`controllers/` (비즈니스 로직까지)	제각각

→ 새로 온 개발자의 첫 주: 어디에 뭐가 있는지 찾기

© 한상훈 NestJS 백엔드 마스터

구조를 정하지 않는 Express

라우터 위치, 서비스 계층 여부

Express와 무관

프로젝트 구조

처음 만든 사람의 습관

그 사람 퇴사 뒤

나눈 이유를 아는 사람 없음

© 한상훈 NestJS 백엔드 마스터

어느 회사든 같은 자리

→ `src/main.ts`, `src/app.module.ts`

→ 기능마다 폴더 하나

→ 폴더 안: 모듈, 컨트롤러, 서비스 파일이 이름 규칙대로

→ 예약 기능 → `reservations/`

```
회사 A
src/
 ├── main.ts
 ├── app.module.ts
 └── reservations/
     ├── reservations.module.ts
     ├── reservations.controller.ts
     └── reservations.service.ts
```

```
회사 B
src/
 ├── main.ts
 ├── app.module.ts
 └── orders/
     ├── orders.module.ts
     ├── orders.controller.ts
     └── orders.service.ts
```

© 한상훈 NestJS 백엔드 마스터

프랜차이즈 주방 배치도

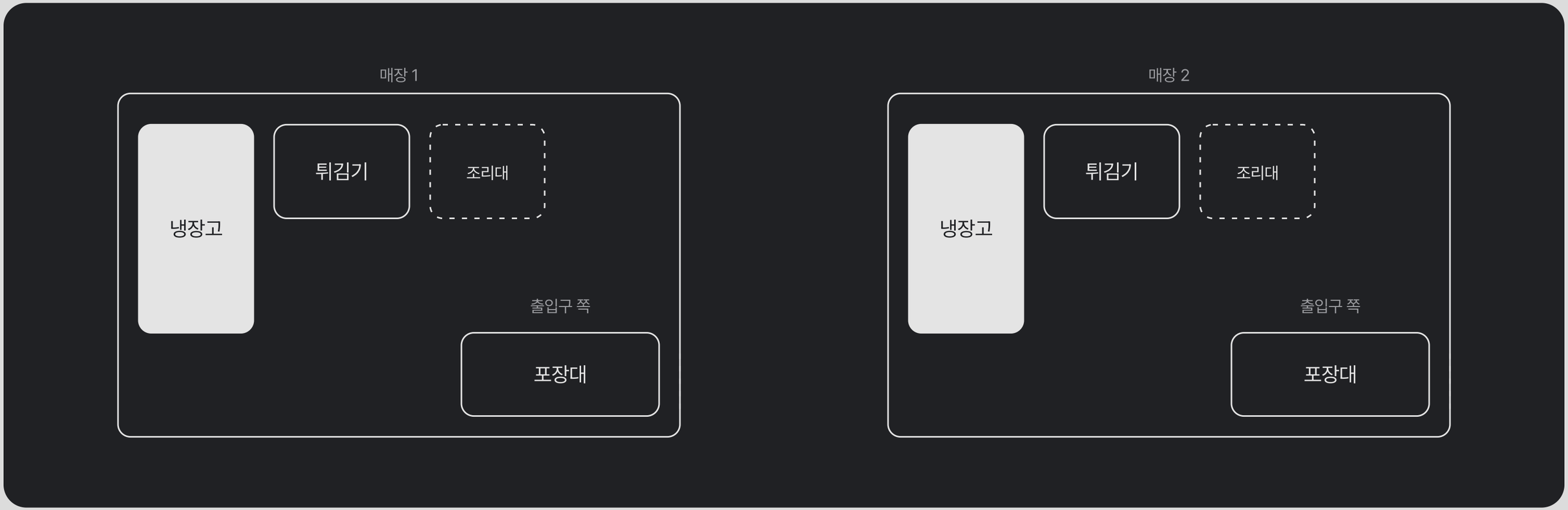
→ 본사가 새 매장마다 같은 주방 배치도

→ 냉장고는 왼쪽 벽, 튀김기는 그 옆, 포장대는 출입구 쪽

→ 다른 매장 직원도 첫날부터 근무

→ Nest CLI = 본사: 프로젝트 생성, 기능 추가 모두 같은 배치도

→ 문서 규칙은 몇 달 뒤 무시, 도구가 만든 파일은 그 자리에



© 한상훈 NestJS 백엔드 마스터

nest new ticketing

터미널

```
$ nest new ticketing
CREATE ticketing/.prettierrc (56 bytes)
CREATE ticketing/nest-cli.json (179 bytes)
CREATE ticketing/oxlint.json (269 bytes)
CREATE ticketing/package.json (1427 bytes)
CREATE ticketing/README.md (6665 bytes)
CREATE ticketing/tsconfig.build.json (178 bytes)
CREATE ticketing/tsconfig.json (603 bytes)
CREATE ticketing/vitest.config.e2e.ts (256 bytes)
CREATE ticketing/vitest.config.ts (363 bytes)
CREATE ticketing/src/app.controller.ts (289 bytes)
CREATE ticketing/src/app.module.ts (265 bytes)
CREATE ticketing/src/app.service.ts (150 bytes)
CREATE ticketing/src/main.ts (245 bytes)
CREATE ticketing/src/app.controller.spec.ts (645 bytes)
CREATE ticketing/test/app.e2e-spec.ts (757 bytes)
✓ Installation in progress... ☕
🚀  Successfully created project ticketing
```

데이터 블록: Nest CLI 12.0.10, 2026-10-25, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

만들어진 파일

루트 설정

package.json, tsconfig.json, tsconfig.build.json, nest-cli.json

루트 도구 설정

린트(oxlint.json), 포매터(.prettierrc), 테스트 (vitest.config.ts)

src/

main.ts, app.module.ts, app.controller.ts, app.service.ts, app.controller.spec.ts

test/

e2e 테스트

데이터 블록: 린트·테스트 도구 종류는 CLI 버전에 따라 다름, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

main.ts

src/main.ts

```ts
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module.js';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
await bootstrap();
```

→ 앱의 시작점: AppModule 하나에서 앱 전체 생성

→ 포트: 환경 변수 PORT, 없으면 3000

© 한상훈 NestJS 백엔드 마스터

모듈, 컨트롤러, 서비스, 데코레이터

모듈

관련된 컨트롤러와
서비스를 묶는 단위

컨트롤러

HTTP 요청을 받는 곳

서비스

실제 일을 하는 곳

데코레이터

클래스 위의 `@Module`,
`@Controller`,
`@Injectable`, 클래스의
역할 표시

© 한상훈 NestJS 백엔드 마스터

app.module.ts와 프로바이더

src/app.module.ts

```ts
import { Module } from '@nestjs/common';
import { AppController } from './app.controller.js';
import { AppService } from './app.service.js';

@Module({
  imports: [],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

→ 프로바이더: providers에 넣은 클래스, Nest가 만들어서 필요한 곳에 주입

→ AppController 생성자의 AppService: Nest가 넣어 줌

© 한상훈 NestJS 백엔드 마스터

ts 파일을 .js로 가져오는 import

파일

```
app.module.ts
```

import 경로

```
'./app.module.js'
```

→ 이유: ES 모듈 설정(package.json `"type": "module"`), 실행되는 파일은 빌드 뒤의 .js

→ .js를 지우면 tsc 빌드 실패: `'./app.module'`을 찾을 수 없음

© 한상훈 NestJS 백엔드 마스터

npm run start와 Hello World!

터미널

```
$ npm run start
LOG [NestFactory] Starting Nest application...
LOG [InstanceLoader] AppModule dependencies initialized +3ms
LOG [RoutesResolver] AppController {/}: +25ms
LOG [RouterExplorer] Mapped {/, GET} route +8ms
LOG [NestApplication] Nest application successfully started +8ms

$ curl localhost:3000
Hello World!
```

→ Hello World!: AppService.getHello()의 반환값

© 한상훈 NestJS 백엔드 마스터

손으로 만드는 공연 기능

1

폴더 생성

2

모듈, 컨트롤러, 서비스
파일 생성

3

`app.module.ts`의
imports에 새 모듈 추가

→ 마지막 단계를 빠뜨리면: 파일은 다 있는데 API 미동작

© 한상훈 NestJS 백엔드 마스터

nest g resource concerts

g

= generate

질문 1: API 방식

→ REST API

질문 2: CRUD 엔드포인트 생성

→ 예

© 한상훈 NestJS 백엔드 마스터

--dry-run

터미널

```
$ nest g resource concerts --dry-run
CREATE src/concerts/concerts.controller.ts (1000 bytes)
CREATE src/concerts/concerts.controller.spec.ts (622 bytes)
CREATE src/concerts/concerts.module.ts (284 bytes)
CREATE src/concerts/concerts.service.ts (683 bytes)
CREATE src/concerts/concerts.service.spec.ts (495 bytes)
CREATE src/concerts/dto/create-concert.dto.ts (34 bytes)
CREATE src/concerts/dto/update-concert.dto.ts (188 bytes)
CREATE src/concerts/entities/concert.entity.ts (25 bytes)
UPDATE package.json (1461 bytes)
UPDATE src/app.module.ts (343 bytes)
Dry run enabled. No files written to disk.
```

→ 실제로 쓰지 않고 만들 파일과 고칠 파일만 표시

© 한상훈 NestJS 백엔드 마스터

만든 파일과 고친 파일

구분	줄 수	성격
CREATE	8	틀에서 찍어 낸 새 파일
UPDATE	2	내가 쓴 기존 파일 수정

© 한상훈 NestJS 백엔드 마스터

app.module.ts diff

```
git diff src/app.module.ts
```

```
 import { AppService } from './app.service.js';
+import { ConcertsModule } from './concerts/concerts.module.js';

 @Module({
-  imports: [],
+  imports: [ConcertsModule],
   controllers: [AppController],
```

© 한상훈 NestJS 백엔드 마스터

package.json diff

```
git diff package.json
```

```
      "@nestjs/core": "^12.0.1",
+     "@nestjs/mapped-types": "*",
      "@nestjs/platform-express": "^12.0.1",
```

→ 이유: `update-concert.dto.ts`의 PartialType

© 한상훈 NestJS 백엔드 마스터

스키매틱

스키매틱(schematic)

파일을 찍어 내는 틀 + 기존 파일 수정 규칙

예

```
nest g resource
nest g controller
nest g service
```

매번 같은 것

파일 이름 규칙, 폴더 위치, 모듈 등록

© 한상훈 NestJS 백엔드 마스터

새로 연결된 라우트 다섯 개

터미널

```
LOG [RouterExplorer] Mapped {/, GET} route
LOG [RouterExplorer] Mapped {/concerts, POST} route
LOG [RouterExplorer] Mapped {/concerts, GET} route
LOG [RouterExplorer] Mapped {/concerts/:id, GET} route
LOG [RouterExplorer] Mapped {/concerts/:id, PATCH} route
LOG [RouterExplorer] Mapped {/concerts/:id, DELETE} route

$ curl localhost:3000/concerts
This action returns all concerts
```

→ DB 없음: 서비스가 문자열만 반환하는 껍데기

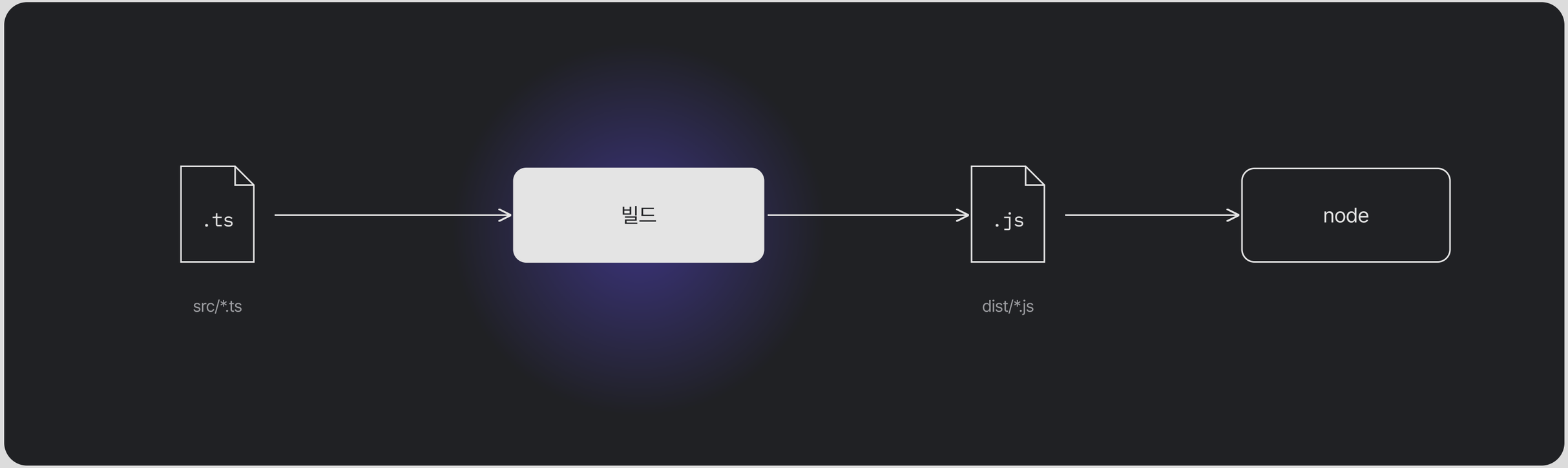
© 한상훈 NestJS 백엔드 마스터

nest start가 하는 일

→ start 스크립트 = `nest start`

→ 빌드 → 빌드 결과를 node로 실행

→ 빌드: .ts → .js (Node는 TypeScript를 그대로 실행하지 않음)



© 한상훈 NestJS 백엔드 마스터

nest build와 dist

터미널

```
$ nest build
$ ls dist
app.controller.d.ts  app.controller.js  app.controller.js.map
app.module.d.ts      app.module.js      app.module.js.map
app.service.d.ts     app.service.js     app.service.js.map
concerts/            main.d.ts          main.js  main.js.map
```

→ .d.ts: 타입 선언

→ .js.map: 소스 맵, dist 줄 번호와 원래 .ts 줄 번호의 대응표 (디버거, 에러 추적 도구가 사용)

© 한상훈 NestJS 백엔드 마스터

tsconfig.json과 tsconfig.build.json

파일	쓰는 곳	차이
`tsconfig.json`	에디터, 테스트	전체 설정
`tsconfig.build.json`	`nest build`	exclude: `test, dist, **/*spec.ts`

→ 테스트 코드는 dist에 포함되지 않음

© 한상훈 NestJS 백엔드 마스터

start, start:dev, start:prod

스크립트	내용	쓰는 때
`start`	`nest start`	빌드 후 실행
`start:dev`	`nest start --watch`	개발, 저장마다 다시 빌드·실행
`start:prod`	`node dist/main`	운영, 미리 빌드한 JavaScript만 실행

→ 컨테이너 이미지: 이미지 안에서 빌드, 실행 명령은 `node dist/main`

→ 하루 수백 번 저장 = 하루 수백 번 빌드

© 한상훈 NestJS 백엔드 마스터

빌더를 SWC로

터미널

```
npm i -D @swc/cli @swc/core
nest build -b swc
```

nest-cli.json

```
{
  "compilerOptions": {
    "deleteOutDir": true,
    "builder": "swc"
  }
}
```

→ SWC: Rust로 만든 TypeScript, JavaScript 컴파일러

→ 한 번만: `-b swc` / 계속: `nest-cli.json`의 builder

© 한상훈 NestJS 백엔드 마스터

tsc와 SWC 컴파일 시간

빌더	컴파일 시간 (파일 10개)	타입 검사
tsc	1.3~1.8초	함
SWC	54~82밀리초	안 함

→ nest build 명령 전체 시간: CLI 시작 시간 포함, 둘 다 수 초

→ start:dev(CLI 상주) 중에는 컴파일 시간 차이가 그대로 체감

데이터 블록: 2026-10-25 강사 PC 실측, tsc는 --extendedDiagnostics Total time, SWC는 CLI 출력, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

number에 넣은 문자열

src/app.service.ts

```ts
const seats: number = '50000';
```

터미널

```
$ nest build
src/app.service.ts:6:11 - error TS2322: Type 'string' is not assignable to type 'number'.
Found 1 error(s).

$ nest build -b swc
>  SWC  Running...
Successfully compiled: 10 files with swc (56.64ms)
```

→ SWC: 타입 표시만 지우고 JavaScript 생성

© 한상훈 NestJS 백엔드 마스터

지운 .js 확장자

터미널

```
// main.ts: import { AppModule } from './app.module';

$ nest build
src/main.ts:2:27 - error TS2307: Cannot find module './app.module' or its corresponding type declarations.

$ nest build -b swc
Successfully compiled: 10 files with swc (58.78ms)
$ node dist/main.js
Error [ERR_MODULE_NOT_FOUND]: Cannot find module '.../dist/app.module' imported from .../dist/main.js
```

→ 빌드 때 잡을 실수를 실행 때 발견

© 한상훈 NestJS 백엔드 마스터

타입 검사를 따로

터미널

```
$ nest build -b swc --type-check
✕  TSC  Initializing type checker...
 Error  Found 1 type error(s) during compilation.
```

```
npx tsc --noEmit -p tsconfig.json
```

→ `--type-check`: SWC 빌드와 타입 검사 동시 실행

→ `tsc --noEmit`: 파일 생성 없이 검사만, 커밋 전이나 CI에서

→ 빌드 통과 ≠ 타입 일치

© 한상훈 NestJS 백엔드 마스터

앱이 하나 더: 알림 워커

예매 완료 문자 발송

예매 API와 분리. 문자 발송 대행사가 느려져도 예매 API는 그대로

같이 쓰는 코드

좌석 표기 'A구역 3열 15번'

모노레포

저장소 하나에 앱 여러 개

→ 저장소 둘이면: 복사, 한쪽만 수정

→ 예: 화면은 '2층 A구역 3열 15번', 복사본을 안 고친 문자는 층 누락 → 고객센터 문의

© 한상훈 NestJS 백엔드 마스터

nest generate app notifier

터미널

```
$ nest generate app notifier
CREATE apps/ticketing/src/main.ts (245 bytes)
CREATE apps/ticketing/src/app.module.ts (343 bytes)
CREATE apps/ticketing/src/concerts/concerts.module.ts (284 bytes)
...
CREATE apps/notifier/tsconfig.app.json (280 bytes)
CREATE apps/notifier/src/main.ts (260 bytes)
CREATE apps/notifier/src/notifier.module.ts (300 bytes)
CREATE apps/notifier/src/notifier.service.ts (155 bytes)
...
UPDATE tsconfig.json (762 bytes)
UPDATE package.json (1541 bytes)
UPDATE nest-cli.json (830 bytes)
```

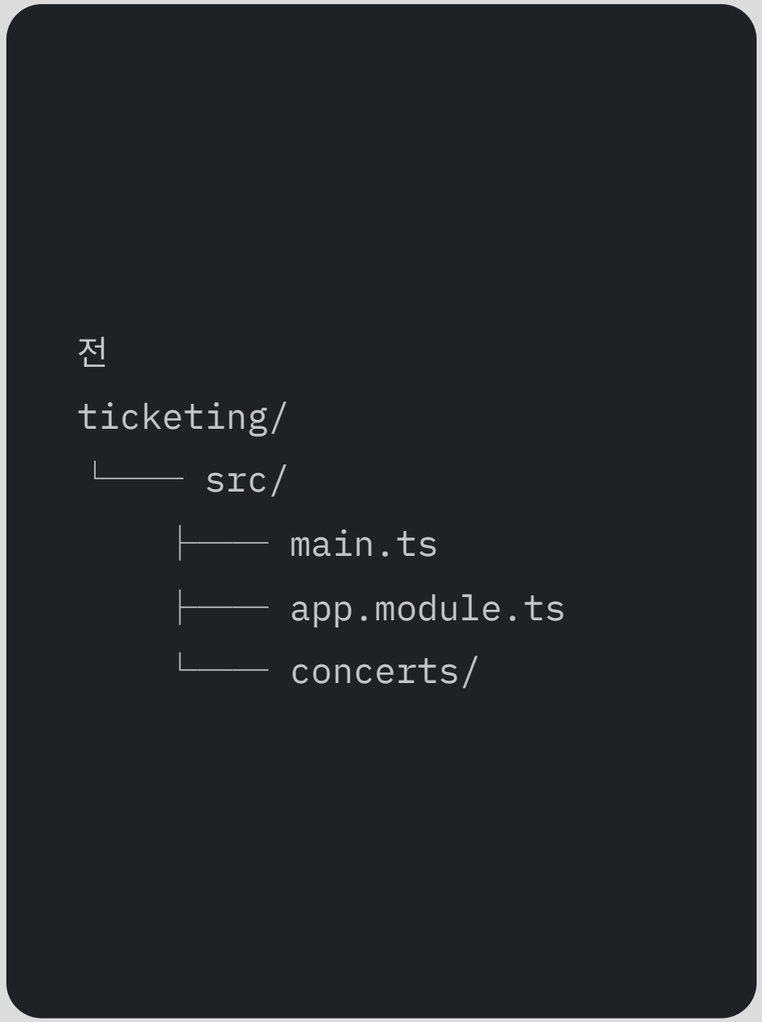
© 한상훈 NestJS 백엔드 마스터

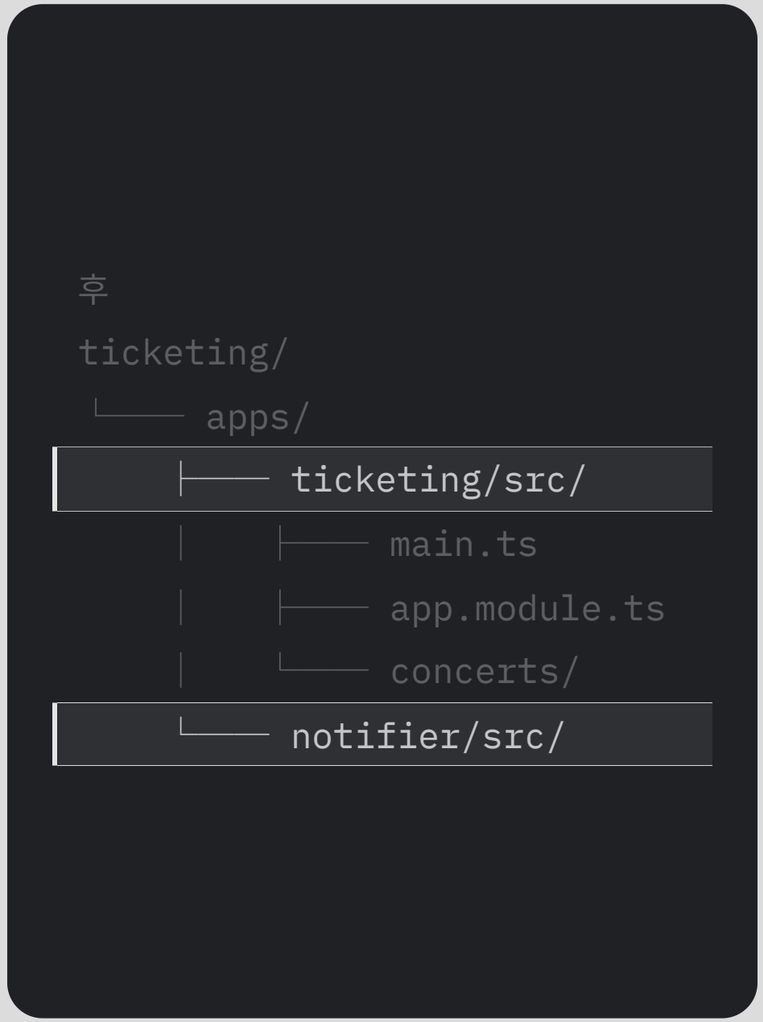
바뀐 폴더 구조

→ 전: src/ 하나

→ 후: apps/ticketing/src/, apps/notifier/src/

→ 기존 코드 전부 apps/ticketing으로 이동





© 한상훈 NestJS 백엔드 마스터

nest-cli.json diff

```
git diff nest-cli.json
```

```
-  "sourceRoot": "src",
+  "sourceRoot": "apps/ticketing/src",
   "compilerOptions": {
-    "deleteOutDir": true
+    "deleteOutDir": true,
+    "builder": "rspack",
+    "tsConfigPath": "apps/ticketing/tsconfig.app.json"
+  },
+  "monorepo": true,
+  "root": "apps/ticketing",
+  "projects": {
+    "notifier": { "type": "application", "root": "apps/notifier", "entryFile": "main", ... },
+    "ticketing": { "type": "application", "root": "apps/ticketing", "entryFile": "main", ... }
   }
```

→ rspack: 여러 파일을 하나로 묶는 번들러

데이터 블록: 모노레포 전환 때 들어가는 builder 값은 CLI 버전에 따라 다름, Nest CLI 12.0.10 기준, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

깨진 test:e2e 줄

```
git diff package.json
```

```
-    "test:e2e": "vitest run --config ./vitest.config.e2e.ts"
+    "test:e2e": "viapps/ticketing/test run --config ./vitest.config.e2e.ts"
```

→ 경로를 바꾸며 vitest 안의 test까지 치환

→ CLI가 기존 파일을 고친 뒤: git diff 확인 후 커밋

데이터 블록: Nest CLI 12.0.10에서 관찰, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

nest g library common

```
nest g library common
```

```
CREATE libs/common/tsconfig.lib.json (278 bytes)
CREATE libs/common/src/index.ts (75 bytes)
CREATE libs/common/src/common.module.ts (203 bytes)
CREATE libs/common/src/common.service.ts (94 bytes)
CREATE libs/common/src/common.service.spec.ts (481 bytes)
UPDATE nest-cli.json (1085 bytes)
UPDATE package.json (1541 bytes)
UPDATE tsconfig.json (907 bytes)
```

```
git diff tsconfig.json
```

```
+    "paths": {
+      "@app/common": ["./libs/common/src"],
+      "@app/common/*": ["./libs/common/src/*"]
+    }
```

© 한상훈 NestJS 백엔드 마스터

두 앱이 같이 쓰는 seatLabel

```ts
// libs/common/src/seat-label.ts
export function seatLabel(zone: string, row: number, no: number): string {
  return `${zone}구역 ${row}열 ${no}번`;
}
```

```ts
// apps/ticketing/src/app.service.ts
import { seatLabel } from '@app/common';
...
    return `예매 API: ${seatLabel('A', 3, 15)}`;
```

```ts
// apps/notifier/src/notifier.service.ts
import { seatLabel } from '@app/common';
...
    return `알림: ${seatLabel('A', 3, 15)} 예매 확정`;
```

© 한상훈 NestJS 백엔드 마스터

앱마다 main.js 하나

터미널

```
$ nest build
Rspack 2.2.9 compiled successfully in 25 ms
$ nest build notifier
Rspack 2.2.9 compiled successfully in 25 ms
$ find dist -name "*.js"
dist/apps/notifier/main.js
dist/apps/ticketing/main.js

$ curl localhost:3000
예매 API: A구역 3열 15번
$ curl localhost:3001
알림: A구역 3열 15번 예매 확정
```

→ 알림 워커 포트: 3001로 변경

© 한상훈 NestJS 백엔드 마스터

번들러도 타입 검사 없음

터미널

```
// seatLabel(zone, row, no, extra) 로 바꾸고 호출은 그대로
$ nest build
Rspack 2.2.9 compiled successfully in 34 ms

$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
apps/ticketing/src/app.service.ts(7,23): error TS2554: Expected 4 arguments, but got 3.
```

→ 앱마다 tsc --noEmit

→ CLI가 만든 설정 그대로는 tsc가 @app/common을 못 찾음, 고친 설정은 부록

데이터 블록: Nest CLI 12.0.10에서 관찰, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

저장소의 첫 구조

```
ticketing/
├── nest-cli.json
├── package.json
├── tsconfig.json
├── node_modules/              앱이 둘이어도 하나
├── apps/
│   ├── ticketing/src/
│   │   ├── main.ts
│   │   ├── app.module.ts
│   │   └── concerts/          module, controller, service, dto, entities
│   └── notifier/src/
└── libs/
    └── common/src/            seat-label.ts
```

→ 공연 API: `apps/ticketing/src/concerts`

→ 문자 발송: `apps/notifier`

→ 같이 쓰는 코드: `libs`

→ 회차 기능 추가 → concerts 옆에 같은 모양의 performances

© 한상훈 NestJS 백엔드 마스터

남은 결정 세 가지

정할 것	이 프로젝트의 결정
빌더	빠른 빌더
타입 검사 위치	`tsc --noEmit` 따로
CLI가 고친 파일 확인	diff 확인 후 커밋

→ CLI가 정해 주는 것: 폴더 이름, 파일 이름, 모듈 등록 방식

© 한상훈 NestJS 백엔드 마스터

첫 커밋

apps/ticketing

예매 API 하나

apps/notifier

알림 워커 하나

libs/common

같이 쓰는
라이브러리 하나

concerts

문자열만 돌려주는
공연 API